

OTP Controller

Design Requirement Specification (DRS)

Revision	Date	Author	Description
1.0	2026-05-13	F. Baharvand	Initial release

Terminology

Term	Definition
OTP (One-Time Programmable) memory	Non-volatile memory that can be programmed once during production. For digital design, it is modelled as a synchronous read-only memory (ROM) with 256 bits total.
OTP Controller	The digital module that reads OTP data at power-on, stores configuration fields in internal registers, and provides these values to other modules (e.g., TC1, TC2, calibration ratios).
Bank0 (Primary)	The main 256-bit OTP array containing the full configuration image.
Patch records	Two optional 28-bit records that can override selected fields in Bank0 if a correction is needed after initial programming.
OTP bit address	A 0-based index from 0 to 255 representing a single bit in the OTP array.
Synchronous ROM	A simulation model of OTP where data is read on the rising edge of the clock when an address is applied. In real silicon, OTP has its own timing; for digital design, a simple ROM model is sufficient.
Field	A group of contiguous bits in OTP that represents a specific parameter (e.g., RO_TRIM_CODE, TC1_COEFF).

1. Purpose

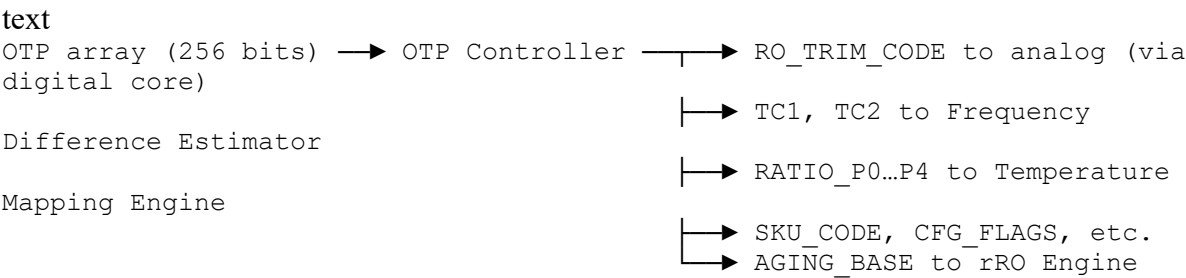
The **OTP Controller** provides a digital interface to read configuration data from the one-time programmable memory and distribute it to other modules. The OTP memory contains 256 bits organized into fields as defined in this document. At power-on reset (POR), the controller reads all necessary fields, applies any patch overrides, and stores the final values in output registers. Other modules (Frequency Difference Estimator, Temperature Mapping Engine, Sigma-Delta Modulator, etc.) then read these registers directly.

Key outcomes:

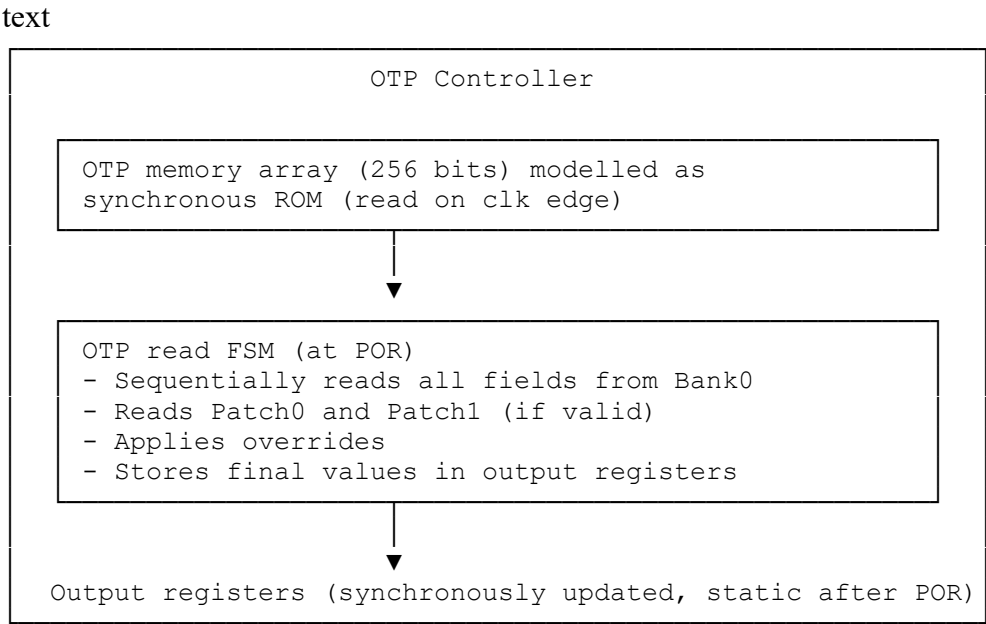
- Simplifies system integration: other modules access configuration data via simple register outputs.
- Supports patch mechanism to correct a small number of OTP bits without reprogramming the entire Bank0.
- The OTP is modelled as a synchronous ROM for RTL simulation; the synthesis target can be a register file or a real OTP macro.

2. Overview

2.1 Position in System



2.2 Block Diagram



2.3 Input/Output Summary

Port	Direction	Width	Description
------	-----------	-------	-------------

clk	Input	1	System clock (RO_CLK, 131.072 kHz)
rst_n	Input	1	Asynchronous reset, active low. Also triggers OTP read at release.
otp_data_out	Input	1	From OTP array (simulated as ROM output). During simulation, this is tied to the ROM output.
otp_addr	Output	8	8-bit address (0...255) to OTP array.
otp_read_en	Output	1	Read enable for OTP (pulsed for one cycle per address).
busy	Output	1	High during OTP read sequence (from POR until all fields loaded).
done	Output	1	Pulses high for one cycle when loading completes.

Output configuration registers (all static after done):

Register	Width	Format	Description
RO_TRIM_CODE	15	unsigned	Main RO trim code (analog)
TC1_COEFF	16	signed	TC1 for Frequency Model (0.25 ppm/°C per LSB)
TC2_COEFF	16	signed	TC2 for Frequency Model (0.01 ppm/°C ² per LSB)
AGING_BASE	16	unsigned Q0.16	Baseline ratio rRO/Main for aging
SKU_CODE	2	unsigned	00:±100ppm, 01:±50ppm, 10:±20ppm
TEMP_RO_TRIM	6	unsigned	Optional trim for Temp RO
REV_ID	6	unsigned	OTP map revision
CFG_FLAGS	6	unsigned	Feature enables (e.g., dither, $\Sigma\Delta$ order, measurement cadence)
RATIO_P0...P4	5×20	unsigned Q4.16	Calibration ratios for –40, –10, 25, 60, 85°C
OE_MODE	1	unsigned	0=OE, 1=PD (from CFG_FLAGS or dedicated field)
OE_POL	1	unsigned	0=active-high, 1=active-low
PROG_DISABLE_LOCK	1	unsigned	1 = OTP programming permanently disabled
PATCH_VALID0,1	2×1	unsigned	Validity bits for patch records

All output registers are **read-only** from the perspective of other modules. They are updated only during the POR load sequence and retain their values until the next reset.

3. Theory of Operation

3.1 OTP Memory Organisation

The OTP is 256 bits total, addressed from 0 to 255 (bit-addressable). The base document defines the exact bit allocation in Section 5.1 (Bank0) and Section 5.2 (Patch records). The following tables summarise the fields (from the base document, pages 21-22).

Bank0 (Primary, bits 0...255, but actual usage up to 198 with CRC)

Bit Range	Field	Bits	Format
[14:0]	RO_TRIM_CODE	15	U15
[30:15]	TC1_COEFF	16	S16
[46:31]	TC2_COEFF	16	S16
[62:47]	AGING_BASE	16	UQ0.16
[64:63]	SKU_CODE	2	U2
[70:65]	TEMP_RO_TRIM	6	U6
[76:71]	REV_ID	6	U6
[82:77]	CFG_FLAGS	6	U6
[102:83]	RATIO_P0	20	UQ4.16
[122:103]	RATIO_P1	20	UQ4.16
[142:123]	RATIO_P2	20	UQ4.16
[162:143]	RATIO_P3	20	UQ4.16
[182:163]	RATIO_P4	20	UQ4.16
[198:183]	CRC16	16	U16
[255:199]	RESERVED / PATCH	57	—

Patch records (bits 196...223 for PATCH0, 224...251 for PATCH1, and 252...255 for RSVD_PATCH)

Each patch record is 28 bits:

Bits within record	Field	Width	Description
[27]	PATCH_VALID	1	1 = patch active, 0 = ignore
[26:22]	PATCH_FIELD_ID	5	Specifies which field to override (0...31)
[21:2]	PATCH_DATA	20	New data for the field (least significant bits)
[1:0]	PATCH_TAG	2	Constant 2'b10 for identification

The mapping from `PATCH_FIELD_ID` to OTP field is defined in the base document (page 22). For simulation, we support the following IDs:

ID	Field	Original width	Patch data width
0	RO_TRIM_CODE	15	15 (use DATA[14:0])
1	TC1_COEFF	16	16 (DATA[15:0])
2	TC2_COEFF	16	16
3	AGING_BASE	16	16
4	TEMP_RO_TRIM	6	6 (DATA[5:0])
5	CFG_FLAGS	6	6

8-12	RATIO_P0 ... P4	20 each	20 (DATA[19:0])
------	-----------------	---------	-----------------

3.2 Power-On Read Procedure

On release of `rst_n` (after POR), the controller performs the following steps:

1. **Reset internal state** – Address counter = 0, all output registers cleared, `busy` = 1, `done` = 0.
2. **Read Bank0 fields** – Sequentially read all bits of Bank0 by issuing `otp_addr` from 0 to 255, reading one bit per clock cycle. For multi-bit fields, accumulate bits into temporary registers. This takes 256 cycles.
3. **Validate CRC** – Compute CRC-16/CCITT over bits [182:0] and compare with stored CRC (bits [198:183]). If mismatch, set an error flag (optional; system may still proceed with fallback defaults). The base document does not mandate action on CRC error; for simplicity, the DRS may ignore CRC or report via status register.
4. **Read Patch0 and Patch1** – After loading Bank0, read the patch record bits (addresses 196...223 for Patch0, 224...251 for Patch1). Decode each record:
 - If `PATCH_VALID` is 1 and `PATCH_TAG` == 2'b10, then override the corresponding field in the temporary registers with `PATCH_DATA` (right-justified to the field width).
 - Patch1 overrides take precedence if both patches target the same field.
5. **Apply overrides** – Write final values to output registers.
6. **Assert done** – Pulse `done` high for one cycle and set `busy` = 0.

After this sequence, the output registers are static until the next reset. The controller does **not** continuously read OTP; it only reads once at power-on.

3.3 Clocking and Timing

- The OTP is modelled as a **synchronous ROM** that delivers the data bit one cycle after the address is applied. The controller uses a simple FSM:
 - Cycle 1: Set `otp_addr` = `addr`, `otp_read_en` = 1.
 - Cycle 2: Sample `otp_data_out` and store the bit, then advance to next address.
- For simulation, a simple register array can be used.
- Total latency from reset release to `done` is approximately 256 (Bank0) + 56 (patches) + overhead ≈ 320 clock cycles. At 131 kHz, this is about 2.4 ms, well within the startup requirement (<2 ms nominal; longer is acceptable if necessary).

3.4 Output Register Behaviour

All output registers are updated **only** when `done` is asserted. Before that, they hold their reset value (0). Other modules should wait for `done` before using configuration values. The system can assert a global `cfg_valid` signal (not part of this module) or simply use `done` as an indicator.

4. Interface Definition

4.1 Port List

Port Name	Direction	Width	Description
clk	input	1	System clock
rst_n	input	1	Async reset, active low. Triggers OTP read on release.
otp_data_out	input	1	Serial data output from OTP memory (simulated ROM)
otp_addr	output	8	Address to OTP (0...255)
otp_read_en	output	1	Read enable, high for one cycle when address is valid
busy	output	1	High during OTP read sequence
done	output	1	High for one cycle after loading complete
ro_trim_code	output	14:0	15 bits
tc1_coeff	output	15:0	16 bits signed
tc2_coeff	output	15:0	16 bits signed
aging_base	output	15:0	16 bits unsigned Q0.16
sku_code	output	1:0	2 bits
temp_ro_trim	output	5:0	6 bits
rev_id	output	5:0	6 bits
cfg_flags	output	5:0	6 bits
ratio_p0...p4	output	5×19:0	20 bits each, Q4.16 unsigned
prog_disable_lock	output	1	From OTP bit [??] – see base document page 21: OTP.PROG_DISABLE_LOCK is programmed as final step; its bit location is not explicitly given but we can assume it is part of CFG_FLAGS or a reserved bit. For DRS, we include it as a separate output.
oe_mode	output	1	From CFG_FLAGS or dedicated field
oe_pol	output	1	From CFG_FLAGS

Note: The exact mapping of `prog_disable_lock` and OE fields should follow the base document. For simplicity, we include them as outputs; the RTL will extract them from the appropriate bit positions.

4.2 Internal Registers (for FSM)

Register	Width	Description
addr	8	Current OTP address (0...255)
state	2	FSM state (IDLE, READ_BANK0, READ_PATCH, APPLY, DONE)
temp_regs	various	Temporary storage for each field during loading

patch0_valid, patch1_valid	1	Decoded patch validity
patch0_id, patch1_id	5	Field IDs
patch0_data, patch1_data	20	Patch data

5. Pseudocode

The following pseudocode describes the OTP controller's behaviour.

```
python
# OTP Controller FSM
# States
STATE_IDLE = 0
STATE_READ_BANK0 = 1
STATE_READ_PATCH = 2
STATE_APPLY = 3
STATE_DONE = 4

# Registers
addr = 0
state = STATE_IDLE
busy = 0
done = 0

# Temporary storage for each field (all zero initial)
tmp_RO_TRIM = 0
tmp_TC1 = 0
tmp_TC2 = 0
tmp_AGING_BASE = 0
tmp_SKU = 0
tmp_TEMP_TRIM = 0
tmp_REV = 0
tmp_CFG = 0
tmp_RATIO = [0]*5    # 5 entries
tmp_CRC = 0

patch0_valid = 0
patch0_id = 0
patch0_data = 0
patch1_valid = 0
patch1_id = 0
patch1_data = 0

# On rising edge of clk
always @(posedge clk or negedge rst_n):
    if not rst_n:
        addr = 0
        state = STATE_IDLE
        busy = 0
        done = 0
        # Clear all temp registers
```

```

    # (reset zero)
else:
    case state:
        STATE_IDLE:
            if not busy and not done:
                busy = 1
                addr = 0
                state = STATE_READ_BANK0

        STATE_READ_BANK0:
            # Read one bit from OTP each cycle
            # We assume a separate always block controls the OTP read
            signal.

            # Here we describe the bit accumulation logic.
            bit = otp_data_out    # sampled one cycle after address set
            # Accumulate into the appropriate field based on addr
            if addr <= 14: # RO_TRIM_CODE bits 0..14
                tmp_RO_TRIM[addr] = bit
            elif addr <= 30: # TC1 bits 15..30
                tmp_TC1[addr-15] = bit
            elif addr <= 46: # TC2 bits 31..46
                tmp_TC2[addr-31] = bit
            # ... continue for all fields (see OTP map)
            # Also compute CRC on the fly if desired
            addr = addr + 1
            if addr == 256:
                state = STATE_READ_PATCH
                addr = 196    # start of Patch0 (bits 196..223)

        STATE_READ_PATCH:
            # Read Patch0 and Patch1 bits (28 bits each)
            # For brevity, assume we read sequentially and decode.
            # After reading both patches, state = STATE_APPLY
            ...
            state = STATE_APPLY

        STATE_APPLY:
            # Apply Patch0 if valid
            if patch0_valid and patch0_id in supported_ids:
                case patch0_id:
                    0: tmp_RO_TRIM[14:0] = patch0_data[14:0]
                    1: tmp_TC1 = patch0_data[15:0]
                    # ... other cases
            # Apply Patch1 similarly (overwrites if same id)
            # Then transfer temp registers to output ports
            ro_trim_code = tmp_RO_TRIM
            tc1_coeff = tmp_TC1
            # ... all outputs
            state = STATE_DONE

        STATE_DONE:
            done = 1
            busy = 0
            state = STATE_IDLE

```

OTP read control (separate always block):


```
python
# OTP read control
otp_read_en = 0
if state in (STATE_READ_BANK0, STATE_READ_PATCH):
    otp_read_en = 1
    otp_addr = addr
```

6. Implementation Requirements (RTL)

6.1 Module Template

```
verilog
module otp_controller (
    input wire      clk,
    input wire      rst_n,
    input wire      otp_data_out,
    output reg [7:0] otp_addr,
    output reg      otp_read_en,
    output reg      busy,
    output reg      done,
    // Output configuration fields
    output reg [14:0] ro_trim_code,
    output reg [15:0] tc1_coeff,
    output reg [15:0] tc2_coeff,
    output reg [15:0] aging_base,
    output reg [1:0] sku_code,
    output reg [5:0] temp_ro_trim,
    output reg [5:0] rev_id,
    output reg [5:0] cfg_flags,
    output reg [19:0] ratio_p0,
    output reg [19:0] ratio_p1,
    output reg [19:0] ratio_p2,
    output reg [19:0] ratio_p3,
    output reg [19:0] ratio_p4,
    output reg      prog_disable_lock, // extracted from OTP
    output reg      oe_mode,
    output reg      oe_pol
);
```

6.2 OTP Bit Field Definitions

The RTL must include a local parameter list mapping each address range to the corresponding field. For maintainability, use `localparam` or a `case` statement inside the accumulation logic.

6.3 Reset Behaviour

- Upon `rst_n = 0`, all output registers are cleared to zero, `busy = 0`, `done = 0`, and the FSM returns to `STATE_IDLE`.
- After `rst_n = 1`, the FSM automatically starts the read sequence (no external trigger needed).

6.4 Simulation Model of OTP

For testbench, a synchronous ROM model is required:

```
verilog
// Simple 256x1 ROM for simulation
reg [255:0] otp_rom;
always @(posedge clk) begin
    if (otp_read_en)
        otp_data_out <= otp_rom[otp_addr];
end
```

The `otp_rom` content can be loaded from a text file using `$readmemh`.

6.5 CRC Handling (Optional)

The base document includes a CRC16 over bits [182:0]. The controller may compute CRC and compare; if mismatch, it may set an error flag. The DRS does not require action on CRC error, but the RTL can include a status register (e.g., `crc_error`) for debug.

7. Testbench Requirements

7.1 Test Procedure

1. **Reset and initial read** – Assert `rst_n` for several cycles, then release. Verify:
 - o `busy` goes high immediately after reset release.
 - o `otp_read_en` pulses for each address from 0 to 255 (and then patches).
 - o After ~320 clock cycles, `done` pulses and `busy` returns low.
 - o All output registers are updated with values from the OTP ROM model.
2. **Correct field extraction** – Load a known OTP image into the testbench ROM (e.g., using `$readmemh`). After the read sequence, verify each output register matches the expected value derived from the ROM image.
3. **Patch override test** – Program the OTP image with a patch record (valid bit = 1, correct tag, field ID = 0). Verify that `ro_trim_code` is overridden by the patch data, and other fields retain their Bank0 values.
4. **Multiple patches** – Program two patches targeting the same field. Verify that Patch1 overrides Patch0.
5. **Invalid patch** – Set `PATCH_VALID = 0` or `PATCH_TAG != 2'b10`. Verify that no override occurs.
6. **CRC error** – (Optional) Corrupt a bit in the ROM and verify that an error flag is set if implemented.

7.2 Testbench Structure

```

verilog
module tb_otp_controller;
    reg clk, rst_n;
    wire otp_data_out;
    reg [255:0] otp_mem;
    // Instantiate DUT
    // Generate clock
    // Load otp_mem from file
    // Drive otp_data_out via a small ROM model
    // Monitor outputs
    // Compare with expected
    // Report pass/fail
endmodule

```

7.3 Pass/Fail Criteria

Test	Condition
Reset	All outputs zero, busy=0, done=0
Read sequence	busy high from after reset until done
Correct field values	Each output matches the ROM content (after patches)
Patch override	The targeted field equals patch_data, others unchanged
Done pulse	done high for exactly one cycle after loading
No double read	After done, no further OTP reads occur until next reset

8. Deliverables

Item	Format	Description
RTL code	Verilog	otp_controller.v – synthesizable
Simulation ROM model	Verilog	otp_sim_rom.v – for testbench use
Testbench	Verilog	tb_otp_controller.v – self-checking
OTP memory map definition	Header file	otp_map.vh with parameterised bit ranges
Documentation	PDF/MD	This DRS

End of DRS for OTP Controller

Appendix A: What is OTP and Why is it Used in Commercial Chips?

One-Time Programmable (OTP) memory is a non-volatile storage that can be written exactly once, typically during factory production, and retains its data for the lifetime of the chip. Unlike flash or EEPROM, OTP cannot be erased or rewritten, making it ideal for storing **per-device calibration data, configuration settings, and security keys** that must not be altered after the chip leaves the factory. In commercial timing chips, OTP holds critical parameters like oscillator trim codes, temperature compensation coefficients (TC1, TC2), and aging baselines that are measured individually for each chip during automated test. The benefits of OTP include:

- **Low cost** – smaller area and simpler manufacturing than flash.
- **High reliability** – no risk of data corruption due to unintended writes or power loss during erase.
- **Power efficiency** – consumes zero static power for storage (unlike some non-volatile technologies).
- **Security** – once programmed, the data cannot be changed by an end-user, preventing accidental or malicious alteration of calibration.

For the digital designer, OTP is modelled as a simple read-only memory (ROM) after production. The OTP controller is responsible for reading this data at power-on and distributing it to the rest of the system.

Patch Mechanism: Purpose and Usage

During production, the main OTP array (Bank0) is programmed with all calibration data. However, if a measurement error occurs or a single bit fails to program correctly, re-programming the entire 256-bit OTP is costly or impossible because OTP cannot be erased. The **patch mechanism** solves this by providing two small, 28-bit one-time programmable records (Patch0 and Patch1) that can **override** specific fields of the main Bank0 after it has been programmed. Each patch record contains a field identifier (which parameter to change), up to 20 bits of new data, a validity flag, and a fixed tag (2'b10) to distinguish it from random unprogrammed bits.

At power-on, the OTP controller first reads the entire Bank0, then reads the two patch records. If a patch is valid and its tag matches, the controller overwrites the corresponding field's value with the patch data. Patch1 takes precedence over Patch0 if both target the same field. This approach adds only 56 bits of OTP area but allows correction of up to two independent fields (e.g., fixing a single trim code or a temperature coefficient) without scrapping the die or requiring a complex re-programming flow. For the digital designer, the patch logic is implemented as a simple post-processing step after loading Bank0: the controller compares each valid patch's field ID and conditionally replaces the temporary register content before finalizing the output registers.